CreditCardPayment.java

Line-by-Line Code Explanation

```
public class CreditCardPayment extends Payment {
```
''' Defines a specific payment type. "extends" links it to the Payment base class, meaning it inherits the ID, amount, and timestamp features automatically.

```
private String cardNumber;
private String cvv;
```
''' Declares variables specific to credit cards. "private" completely isolates this data so no other class can see or leak the credit card numbers.

```
public CreditCardPayment(double amount, String currency, String cardNumber... ) {
```
''' The Constructor that takes in both the generic data (amount) and the specific data (cardNumber).

```
super(amount, currency);
```
''' Passes the generic amount and currency UP to the parent Payment class constructor so it can generate the Transaction ID and Timestamp.

```
this.cardNumber = cardNumber;
this.cvv = cvv;
```
''' Saves the specific credit card details into this object's private memory.

```
public String maskedNumber() { ... }
```
''' A utility function that hides most of the card number, returning only the last 4 digits (e.g., •••• 1234) for safe logging.

```
@Override
public boolean process(BiConsumer<String, String> log) {
```
''' The "@Override" tag tells Java we are fulfilling the abstract contract from the parent class. This is the custom algorithm for Credit Cards.

```
String digits = cardNumber.replaceAll("\\s", "");
```
''' Removes all spaces from the credit card string so we can count the pure digits.

```
boolean validCard = digits.length() >= 12 && cvv.matches("^\\d{3}$");
```
''' Validates the card: ensures it is at least 12 digits long, and uses a Regular Expression (^\d{3}$) to verify the CVV is exactly 3 numbers.

```
if (!validCard) {
    this.status = "FAILED";
    return false;
}
```
''' If validation fails, it reaches into the inherited variables, changes the status to FAILED, and instantly stops the method.

```
boolean authorized = amount <= 500_000;
```
''' Simulates a network connection to a bank. We implemented a fake rule that any transaction over $500,000 gets declined.

```
if (authorized) {
    this.status = "SUCCESS";
    return true;
}
```
''' If the bank authorizes it, we update the status to SUCCESS and return true back to the PaymentProcessor.